THE해커톤 : CAMPUS

THE 해커톤

KOREA UNIVERSITY

고대, 만드는 사람들

고대에서 시작해, 세상에 내놓을 것을 만듭니다

2026.10.05 18:30 · 신입 구성원 오리엔테이션

30시간의 몰입이,
출시와 인수로 이어졌습니다

30H

실제로 만들고 내놓은 시간

실제 앱 출시

참가자는 30시간 동안 만든 iOS 앱의 출시를 공개했습니다.

제품 인수 경험

백3팀 참가자는 해커톤에서 만든 제품의 인수를 공개했습니다.

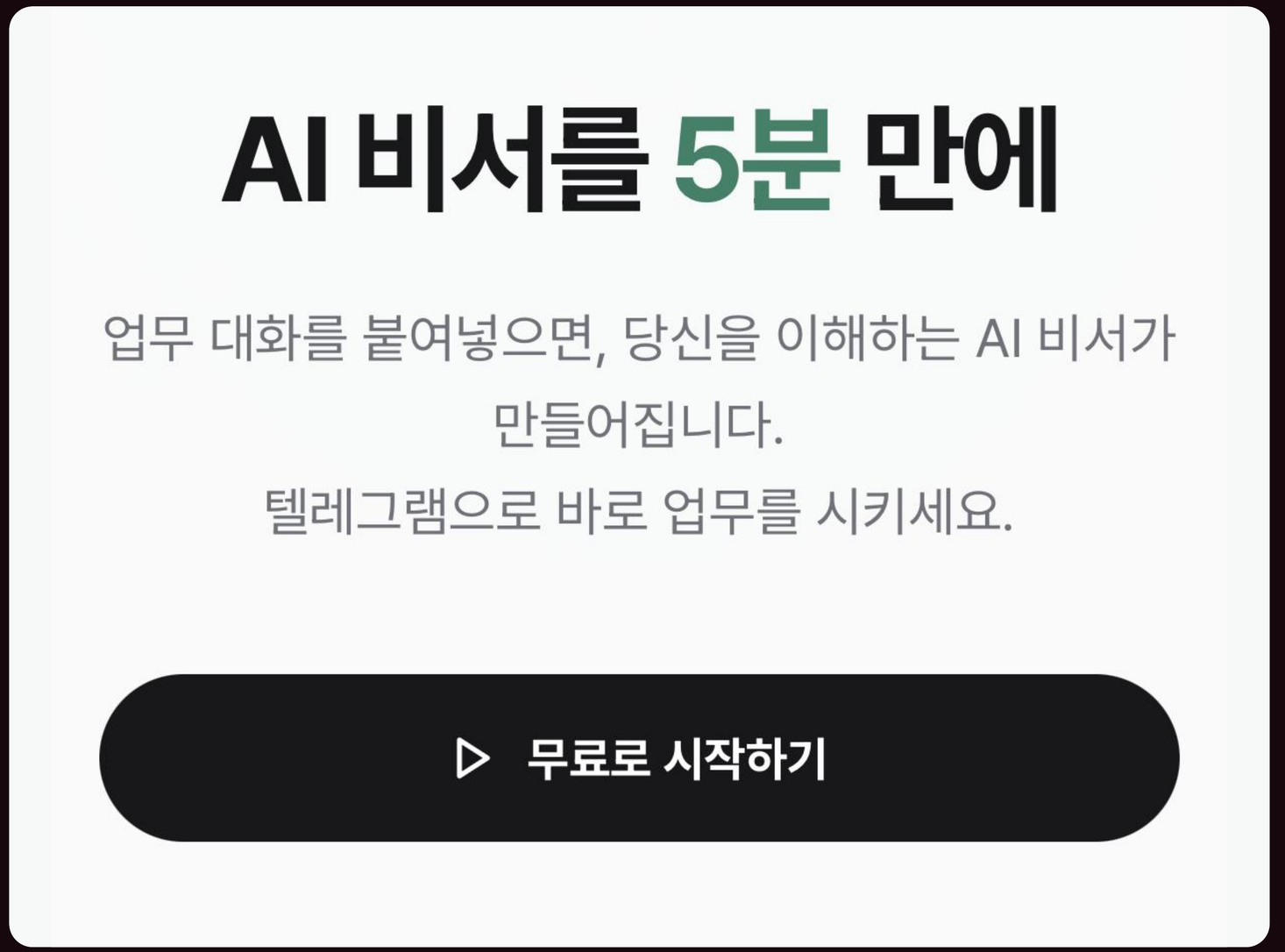
근거: 참가자 출시 후기 ↗

근거: 백3팀 참가 후기 ↗

AUTOME

업무 대화로 나를 이해하는 AI 비서를 만듭니다

평소 업무 대화와 스타일을 분석해,
맞춤형 비서를 설정하는 서비스.

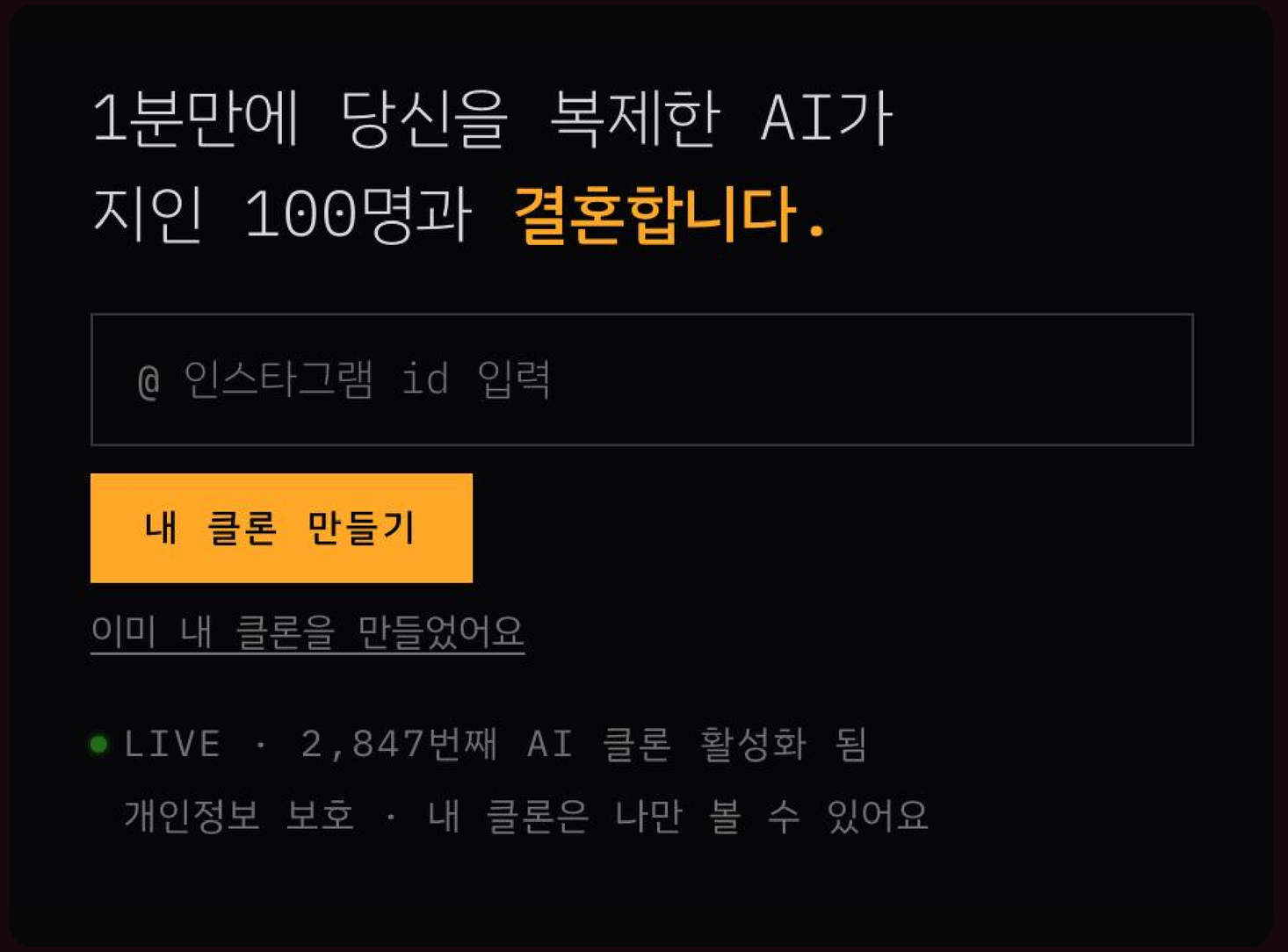


핵심 화면 확대 · 흑백개발자 당시 결과물 소개 자료

LOVE VIRTUALLY

내 프로필로 만든 AI 클론들이 서로 대화합니다

프로필을 바탕으로 만든 클론의
대화와 가상 관계를 체험하는 서비스.

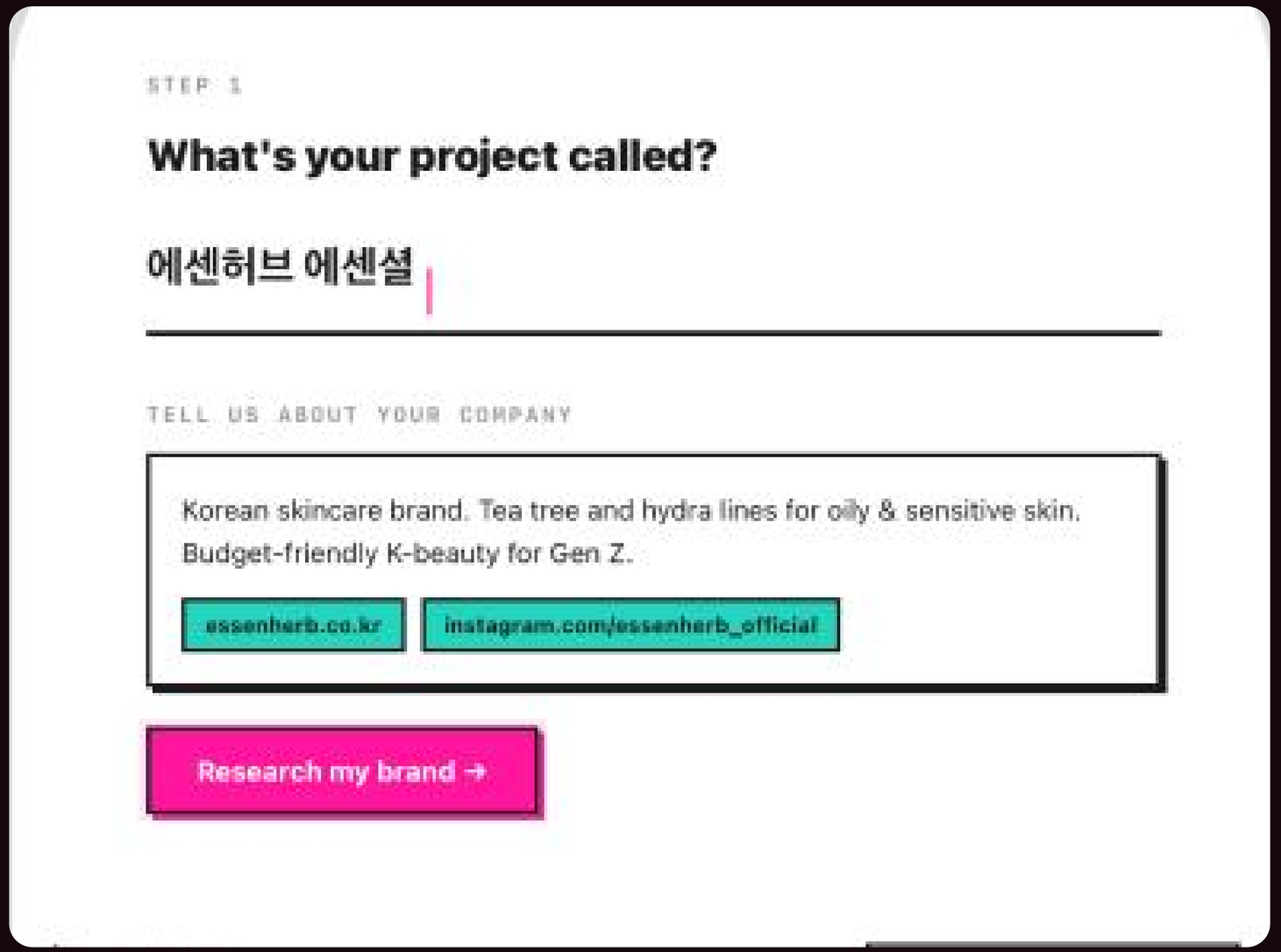


핵심 화면 확대 · 흑백개발자 당시 결과물 소개 자료

KONTI · 브랜드 입력

프로젝트명과 브랜드 정보를 먼저 입력합니다

회사 소개와 브랜드 단서를 입력해
콘텐츠 제작의 기준을 정합니다.

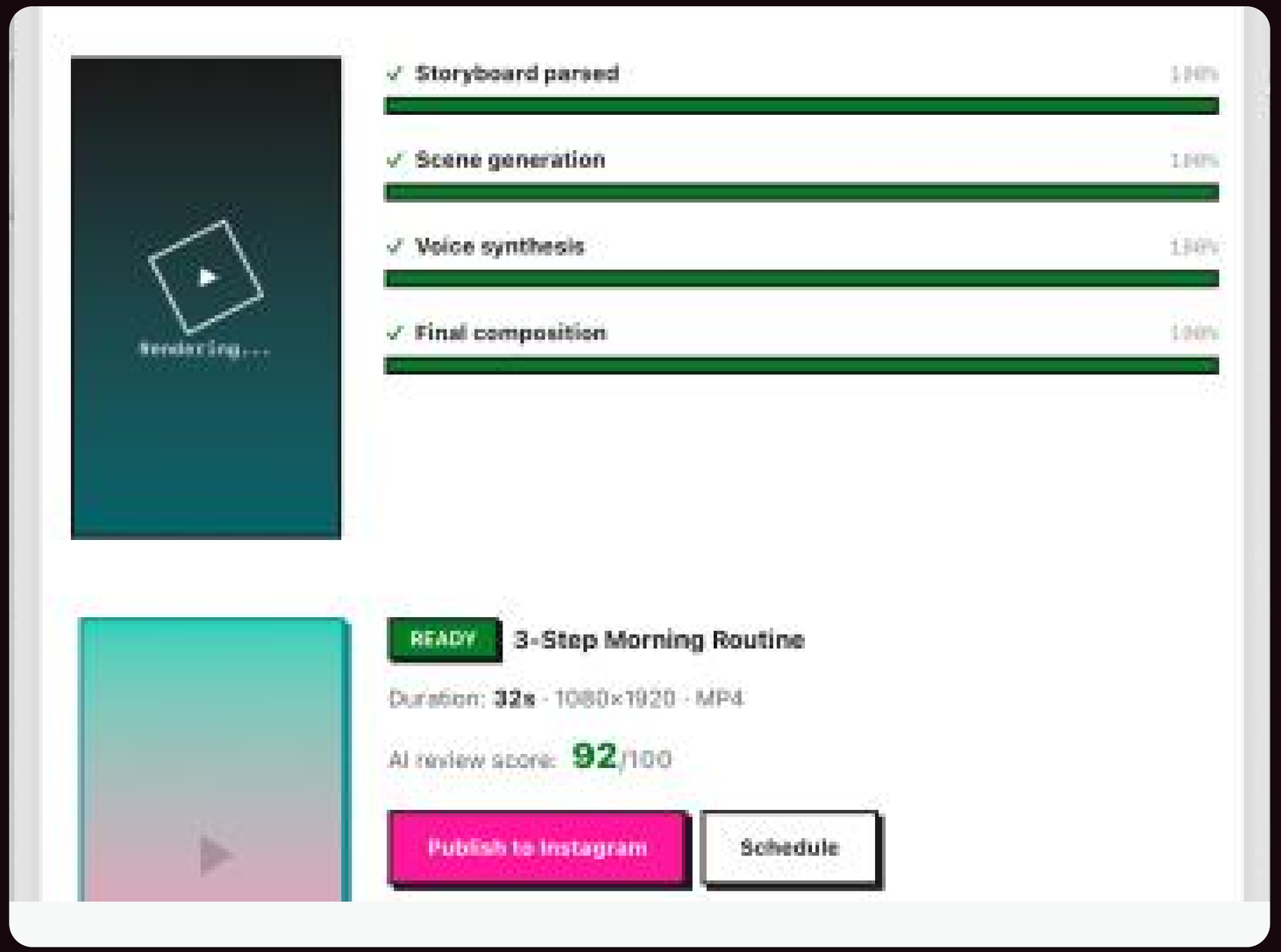


핵심 화면 확대 · 흑백개발자 당시 결과물 소개 자료

KONTI · 제작 결과

입력한 브랜드가 실제 콘텐츠로 이어집니다

제작 진행 상태와 결과물을 확인하고,
게시 단계까지 연결합니다.



핵심 화면 확대 · 흑백개발자 당시 결과물 소개 자료

KOREA UNIVERSITY · BUILDERS

고대에서 만들고,
캠퍼스 밖에서 증명합시다

흑백개발자의 다음 이야기를
우리의 팀과 프로젝트로 이어갑니다.

이번 학기에 남길 것

01

내가 만든 서비스

아이디어를 실제로 쓸 수 있는 제품으로 만들고, 사용자에게 보여줍니다.

02

함께 뛰는 팀

고민만 나누는 데서 멈추지 않고, 마감까지 함께 만드는 동료를 만납니다.

03

설명할 수 있는 성과

무엇을 만들고, 어떤 반응을 얻고, 어떻게 바꿨는지 기록으로 남깁니다.

CAMPUS가 지향하는 경험 · 결과는 우리가 선택한 문제와 실행으로 만듭니다.

THE해커톤 · CAMPUS · DREAMERS

01

THE해커톤

실전 해커톤과 학교 간 연결, 외부 협력의 기회를 설계합니다.

02

CAMPUS

학교별 구성원이 정기 모임과 프로젝트를 주도하는 대학생 활동입니다.

03

DREAMERS

배운 것을 청소년에게 나누는 AI 교육·멘토링 활동입니다.

학교 안의 운영과 연합 활동을 연결하되, 각 프로그램의 모집·참여 조건은 따로 안내합니다.

학기 활동의 두 축

01

정규 프로젝트

작은 해커톤으로 기획·구현을 경험하고, 팀 프로젝트로 서비스를 발전시킵니다.

02

교육·멘토링

AI로 문제를 푸는 과정을 정리하고, 멘티가 스스로 만들 수 있도록 돕습니다.

연대 운영 자료를 바탕으로 한 구성안입니다. 고대의 회차·요일·참여 방식은 별도 확정합니다.

아이디어를 서비스로

01

Pre-HIT

짧은 해커톤으로 문제 정의, 팀 작업, 프로토타입 제작을 반복합니다.

02

HIT

한 가지 문제를 골라 여러 회차에 걸쳐 구현·사용자 확인·개선을 이어갑니다.

03

결과 공유

실행 가능한 링크와 시연, 배운 점을 정리해 활동 기록으로 남깁니다.

프로젝트의 크기보다, 직접 실행하고 설명할 수 있는 결과를 만듭니다.

배움을 나누는 멘토링 : DREAMERS

01

현업 멘토

실무 경험과 문제 해결 관점을 대학생 멘토에게 나눕니다.

02

대학생 멘토

내용을 직접 익히고, 멘티 눈높이에 맞는 설명과 실습을 준비합니다.

03

청소년 멘티

직접 질문하고 만들어 보며, 자신의 문제를 풀어봅니다.

멘토 참여 일정·교육·역할은 별도 모집과 운영 안내를 따릅니다.

고려대 운영과 안내

KU

안희성

고려대학교 경영 26학번
CAMPUS 상담 진행 · 고대 OT 운영 안내

THE

최동인

THE해커톤 소개
학교 지부와 연합 활동의 관계 안내

정규 모임·신청 절차는 고대 운영 안내를 확인해 주세요.

개발 교육 운영진

김민서	컴퓨터학과
김태호	컴퓨터학과
안희성	경영학과

학회원 소개

고세영

인공지능학과

학회원 소개

김도현

경영학과

학회원 소개

고우민

스마트보안학부

학회원 소개

김경범

스마트보안학부

김상후

컴퓨터학과

학회원 소개

김재우

전기전자공학과

학회원 소개

문정우

컴퓨터학과

학회원 소개

선종하

스마트보안학과

학회원 소개

송지석

인공지능학과

이용현

인공지능학과

학회원 소개

정도윤

컴퓨터학과

학회원 소개

정민재

인공지능학과

학회원 소개

진의찬

스마트보안학부

학회원 소개

황윤하

스마트보안학부

학회원 자기소개

01

나는 누구인가

이름 · 전공 · 학번
지금 관심 있는 분야를 소개해 주세요.

02

무엇이 궁금한가

이번 학기에 배우거나 해결해 보고 싶은 문제는 무엇인가요?

03

무엇을 만들 것인가

앱 하나를 바로 만들 수 있다면, 누구의 어떤 불편을 해결하고 싶나요?

각자 1분씩 소개합니다. 잘하는 것과 도움받고 싶은 것도 함께 이야기해 주세요.

오늘 마칠 준비

01	프로필 공유	운영진이 안내한 보드에 자기소개 작성
02	개발 환경	Cursor 설치 후 실행 확인
03	GitHub	계정 준비 및 팀 저장소 참여 방법 확인
04	대화방과 공지	고대 공식 대화방·자료 링크 확인

가입·결제·설치 권한이 필요한 항목은 운영진 안내를 먼저 확인해 주세요.

앞으로의 일정

10/5	고려대 OT	18:30 · 활동 소개와 시작 준비
10/9	전체 발대식 검토	개최 여부·시간·장소 확정 후 별도 공지
추후	첫 정규 세션	일정·장소·참여 방식은 고대 운영진 안내

정규 회차·회비·납부 방식은 확정된 운영 공지를 기준으로 확인해 주세요.

THE해커톤 : CAMPUS

THE 해커톤

이제, 고대의 차례입니다

다음으로 보여줄 결과물은, 여러분 것입니다

고려대학교 OT · Q&A

자연어로 만들고, 직접 확인한다

만들고 싶은 서비스와 동작을 자연어로 설명하고, AI와 함께 구현합니다.

한 번에 모든 기능을 요청하기보다, 작은 기능부터 실행하며 수정합니다.

무엇을 만들지 정하는 일과 결과가 맞는지 확인하는 일은 우리에게 남습니다.

화면이 보이는 것만으로 끝내지 않고, 저장·처리·배포까지 실제로 동작하는지 확인합니다.

오늘은 '프로젝트 제출함'을 예시로 웹 서비스의 구조를 살펴봅니다.

실습 예시: 프로젝트 제출함

01

입력한다

팀 이름과 프로젝트 배포 URL을 입력하고 제출합니다.

02

목록에 남는다

제출한 팀과 링크가 목록에 표시되고, 새로고침해도 남습니다.

03

중복을 확인한다

같은 팀이 다시 제출하면, 중복 안내를 보여줍니다.

이 흐름을 기준으로 화면·처리·저장이 각각 무엇을 하는지 구분합니다.

웹 서비스의 다섯 구성요소

01	프론트엔드	사용자가 보고 입력하는 화면
02	백엔드	요청을 확인하고 처리하는 규칙
03	데이터베이스	새로고침해도 남아야 할 데이터
04	배포	다른 사람도 접속할 수 있게 공개
05	GitHub	변경 기록을 남기고 팀과 공유

화면 → 처리 → 저장이 연결되어야 서비스가 동작합니다.

첫 화면과 데이터 저장

01

먼저, 화면

팀 이름·URL 입력칸, 제출 버튼, 제출 목록을 만듭니다. 먼저 흐름을 눈으로 확인합니다.

02

다음, 저장

제출 내역을 데이터베이스에 저장합니다. 새로고침하거나 다시 접속해도 남는지 확인합니다.

입력칸이 보이는 것과, 실제 제출이 저장되는 것은 별개입니다.

중복 제출을 막는 처리

01

요청

사용자가 제출 버튼을 누르면, 입력한 팀과 링크를 보냅니다.

02

판단

이 팀이 이미 제출했는지 확인합니다. 중복이면 이유를 안내합니다.

03

저장

처리 조건을 통과한 제출만 저장하고, 결과를 화면에 보여줍니다.

‘한 팀은 한 세션에 한 번 제출’은 이 실습에서 사용할 예시 규칙입니다.

내 노트북에서 인터넷으로

01

로컬 실행

내 컴퓨터에서 먼저 실행해 봅니다. localhost 주소는 다른 사람의 접속 주소가 아닙니다.

02

외부 공개

배포한 주소로 다른 사람도 접속하게 합니다. 입력·저장·조회가 모두 되는지 다시 확인합니다.

배포 예시 도구: Vercel · 실제 제출 링크는 운영진 안내에 따릅니다.

수정과 되돌리기

01

작동하는 상태를 기록

기능이 동작할 때마다 무엇을 바꿨는지 기록합니다. 팀원이 이해할 수 있게 설명합니다.

02

변경 뒤 다시 확인

색을 바꿨는데 버튼이 사라졌다면, 변경 전 상태와 비교하고 필요한 수정만 되돌립니다.

작은 변경 → 실행 확인 → 기록을 반복하면, 오류가 생긴 지점을 찾기 쉽습니다.

GitHub 기본 용어

01	Repository · 저장소	프로젝트 파일과 변경 기록을 모으는 곳
02	Commit · 변경 기록	어떤 작업을 했는지 메시지와 함께 기록
03	Push · 올리기	내 변경 기록을 원격 저장소에 전송
04	README · 안내문	실행 방법·사용법·배포 주소를 설명
05	Branch · 작업 분리	각자의 변경을 분리해서 진행

처음부터 모든 기능을 익힐 필요는 없습니다. 기록하고 공유하는 흐름부터 시작합니다.

팀의 코드를 함께 바꾸는 순서

01

분리해서 작업

자기 작업 범위를 정하고, 다른 사람의 작업과 구분해 변경합니다.

02

검토를 요청

Pull Request로 무엇을 바꿨는지 공유하고, 팀원이 확인할 수 있게 합니다.

03

확인 후 합치기

동작과 변경 내용을 확인한 뒤, 공통 코드에 합칩니다. 이를 merge라고 합니다.

역할과 작업 범위를 먼저 맞추면, 서로의 변경을 덮어쓰는 일을 줄일 수 있습니다.

첫 실습에서 확인할 것

01	입력과 제출	팀 이름과 URL을 입력해 실제 제출되는가
02	저장과 조회	새로고침해도 제출 내역이 남는가
03	중복 처리	같은 팀의 재제출이 규칙대로 처리되는가
04	외부 접속	다른 컴퓨터에서도 배포 주소로 접속되는가
05	기록과 인계	저장소·README·실행 방법을 팀이 아는가

시연 화면만 보지 말고, 실제 동작으로 확인합니다.

실습 후 더 살펴볼 주제

01	기획	PRD · 최소 기능 범위 · 사용자 시나리오
02	AI에게 설명하기	요구사항 · 작업 맥락 · 오류 재현 방법
03	화면과 데이터	컴포넌트 · API · 인증 · 데이터 구조
04	검증과 배포	브라우저 도구 · 테스트 · 배포 설정
05	협업	README · 변경 기록 · PR 검토

첫 실습에서 막힌 문제를 기준으로 필요한 주제부터 하나씩 익힙니다.